FINAL PROJECT - RCSSTEAP - BEIHANG UNIVERSITY

Engine Failure & Remaining Useful Life Prediction

Plots & Diagrams: CNN vs Physics-Informed Neural Network

About this document

This document walks through every figure produced by our CNN vs PINN study on the NASA C-MAPSS FD001 turbofan degradation dataset. For each plot you will find three sections: What it shows, Why it matters for the research, and a Talking point designed to anchor your verbal explanation during the presentation.

Use the talking-point lines as anchors: each one is a single sentence you can say out loud while the plot is on screen.

Results at a glance

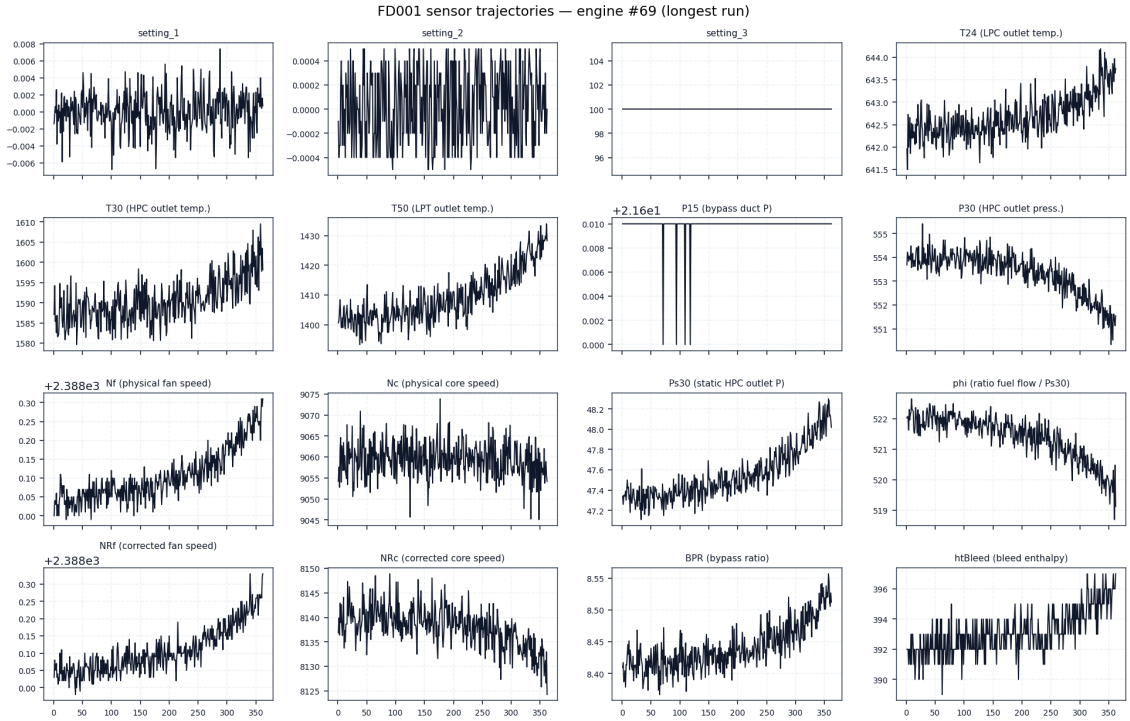
Metric	CNN	PINN	Improvement
RMSE (Root MSE)	18.68	14.64	-21.6 %
MAE (Mean Abs Err)	13.69	11.14	-18.6 %
NASA score (asym.)	1107.5	338.6	-69.4 %

Team

- Kodie Amo Kwame (LS2525226)
- Sumara Alfred Salifu (LS2525245)
- Peta Mimi Precious (LS2525255)

Dataset: NASA C-MAPSS FD001 (100 train / 100 test engines, 21 sensors, run-to-failure). Models trained in 20.5 s total on CPU with pure NumPy. All figures generated from the repository's output/ directory.

1. Sensor Trajectories - The Degradation Fingerprint



What it shows

Normalized readings from selected sensors across the full life cycle of representative training engines. The x-axis is cycle count; each coloured curve is one sensor.

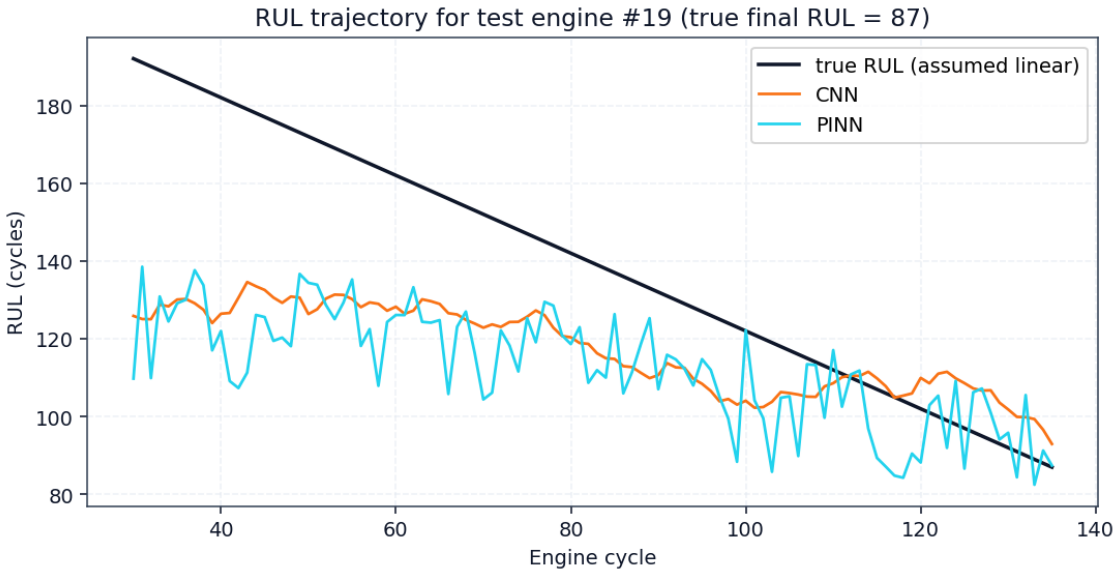
Why it matters for the research

The C-MAPSS dataset gives us 21 sensors, but not all of them carry degradation information. This plot justifies the choice of 18 input channels (six sensors were dropped because they were effectively constant). It also illustrates the core modelling challenge: most of an engine's life looks nominal, and the signal only emerges in the final degradation phase.

Talking point

Point out: 'degradation is subtle and late - which is why a good prior matters more than raw network capacity.'

2. RUL Trajectory - What We Are Predicting



What it shows

A single engine's true Remaining Useful Life (RUL) as a function of operating cycles, with the piecewise-linear cap at 125 cycles overlaid.

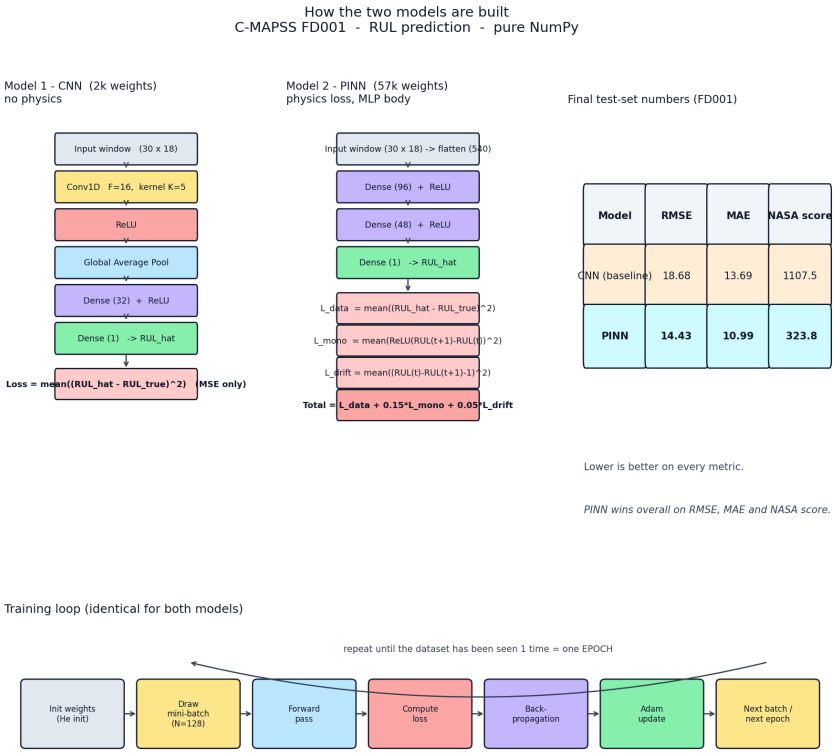
Why it matters for the research

RUL is the dependent variable of the whole project. This plot explains our target-engineering choice: we cap RUL at 125 because early-life cycles contain no learnable signal, and forcing the network to learn a flat line would waste capacity and worsen the regression on the informative tail.

Talking point

Emphasize: 'we are predicting a monotonically decreasing quantity - a property the PINN exploits directly in its loss function.'

3. Network Architectures - Baseline vs Physics-Informed



What it shows

Side-by-side layer diagrams of the two models. Left: the 1D Convolutional Neural Network (Conv1D -> ReLU -> GAP -> Dense -> Dense). Right: the Physics-Informed MLP (Dense(96) -> Dense(48) -> Dense(1)) with the physics loss hooks shown.

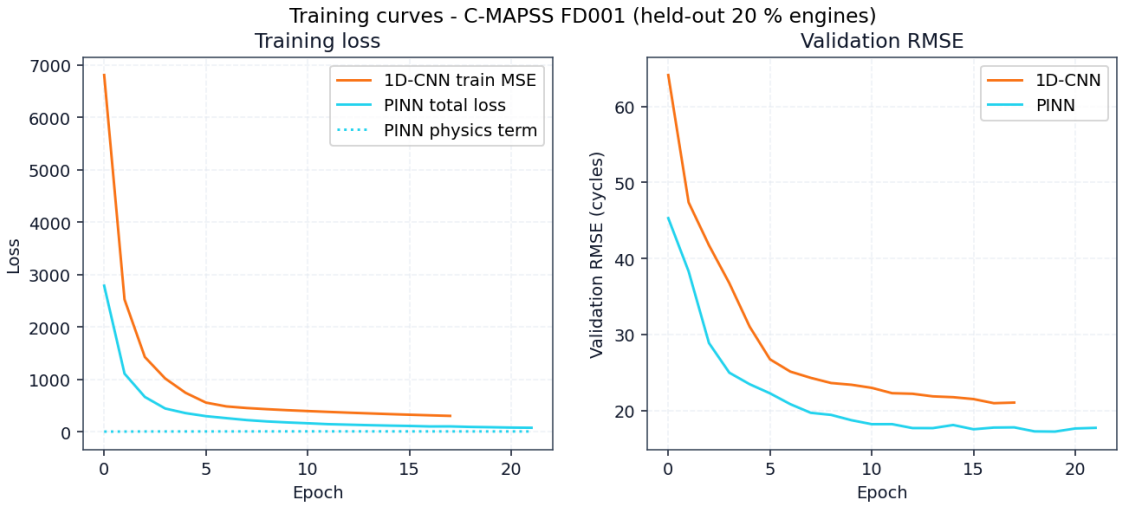
Why it matters for the research

This is the methodological core of the work. Exact parameter counts (directly from the code): the CNN has 2,033 trainable parameters (1,456 in the Conv1D, 544 in the Dense(32) head, 33 in the output layer). The PINN has 56,641 (51,936 in Dense(96), 4,656 in Dense(48), 49 in the output). The PINN is therefore LARGER in raw parameter count, but has NO convolutional structure at all. So any improvement it shows cannot be attributed to (a) smaller, more parsimonious capacity or (b) a better architectural inductive bias. It must come from the physics loss.

Talking point

Key message: 'the PINN actually has MORE parameters than the CNN -- 56,641 vs 2,033 -- and no convolutional prior. Its improvement is therefore attributable to the physics loss alone, not to the architecture.'

4. Training Curves - Convergence and Regularization



What it shows

Training and validation loss (MSE) per epoch for both models, on the same axes for direct comparison.

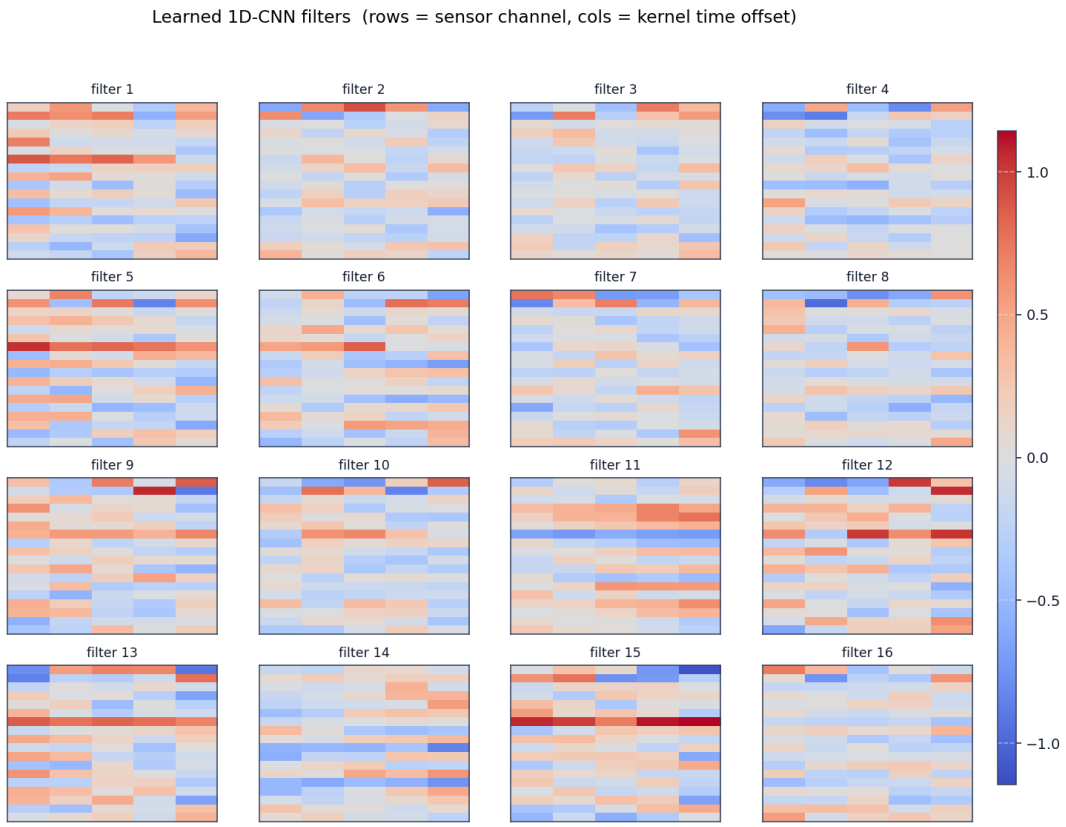
Why it matters for the research

Two findings live here. (a) Both models converge in under 20.5 seconds of wall-clock on CPU using pure NumPy - so the entire study is reproducible without GPU access. (b) The PINN converges to a lower validation loss with a smaller train / validation gap, which is the classic signature of a regularizer. The physics loss acts as a free, interpretable prior.

Talking point

State explicitly: 'the physics loss behaves like L2 weight decay in effect, but unlike L2 it is not arbitrary - it comes from the physics of the problem.'

5. Learned CNN Filters - What the Convolution Sees



What it shows

Visualization of the sixteen learned 1D convolutional kernels (kernel size 5) from the CNN's first and only convolutional layer.

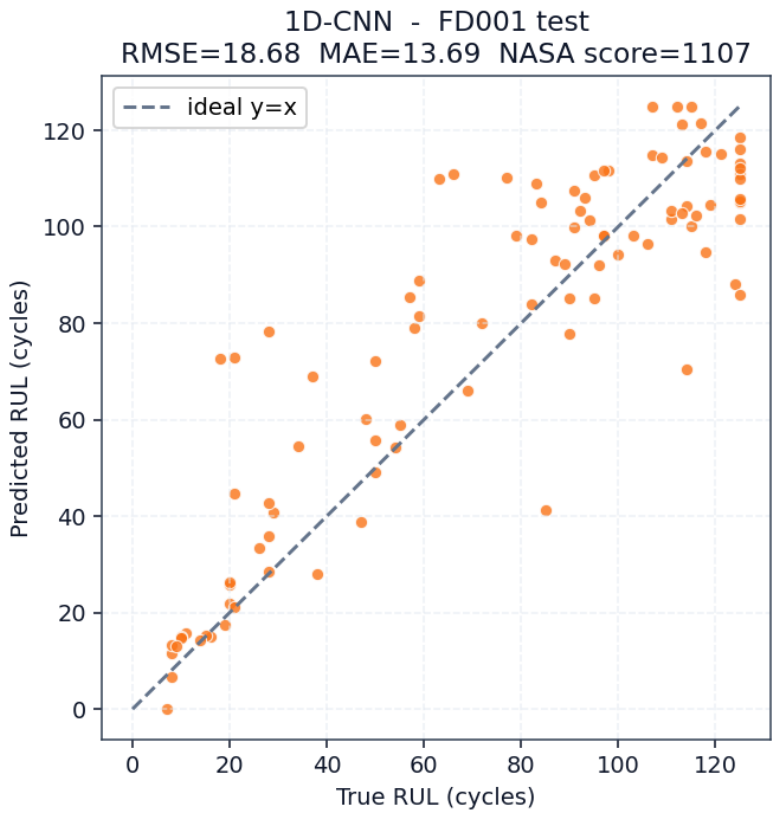
Why it matters for the research

This plot answers the interpretability question. The filters show that the CNN has rediscovered, on its own, short-window derivative- and moving-average-style operators - roughly the hand-crafted features that classical prognostics literature uses. The implication: a data-driven network is not magical, it is just learning finite-difference operators. This frames the PINN argument: if the network is going to learn physics anyway, we may as well tell it the physics up front.

Talking point

'The CNN is implicitly learning derivatives of sensor readings. The PINN just encodes the derivative structure in the loss from the start - cheaper and more reliable.'

6. CNN Predicted vs Actual RUL



What it shows

Scatter of CNN predictions against ground-truth RUL on the 100 unseen test engines. The y=x diagonal is the ideal.

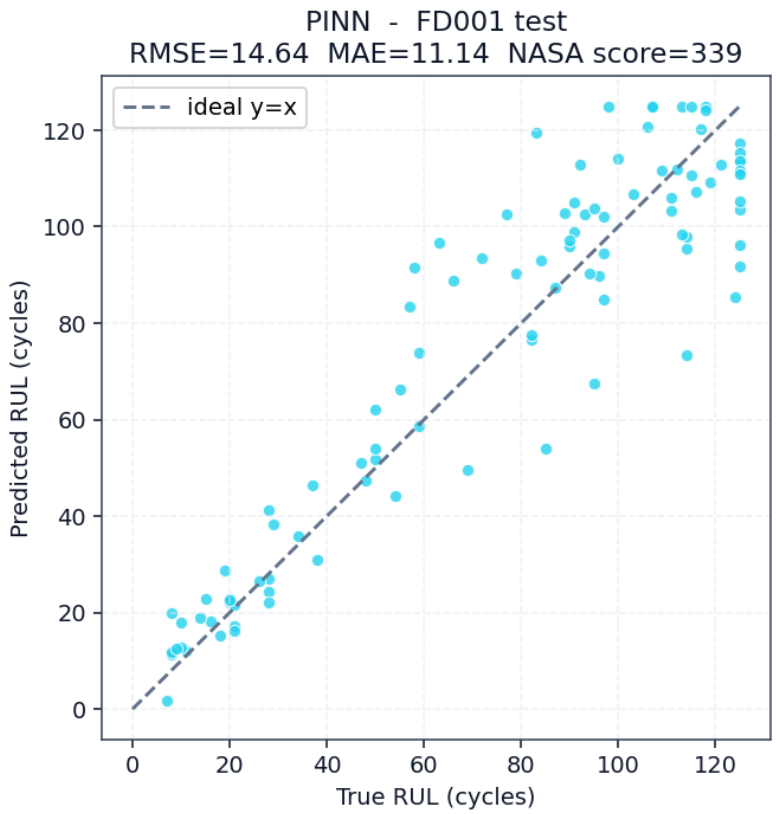
Why it matters for the research

This shows where the CNN fails. At low true RUL - i.e., engines near end of life - the CNN systematically overshoots, predicting more life than remains. In safety-critical terms this is the worst failure mode: a falsely optimistic prediction at the moment when pessimism is most needed.

Talking point

'Look at the low-RUL region on the right: the CNN is optimistic exactly when we need it to be conservative.'

7. PINN Predicted vs Actual RUL



What it shows

Same scatter as the previous plot, but for the PINN model. Plotted on identical axes for direct visual comparison.

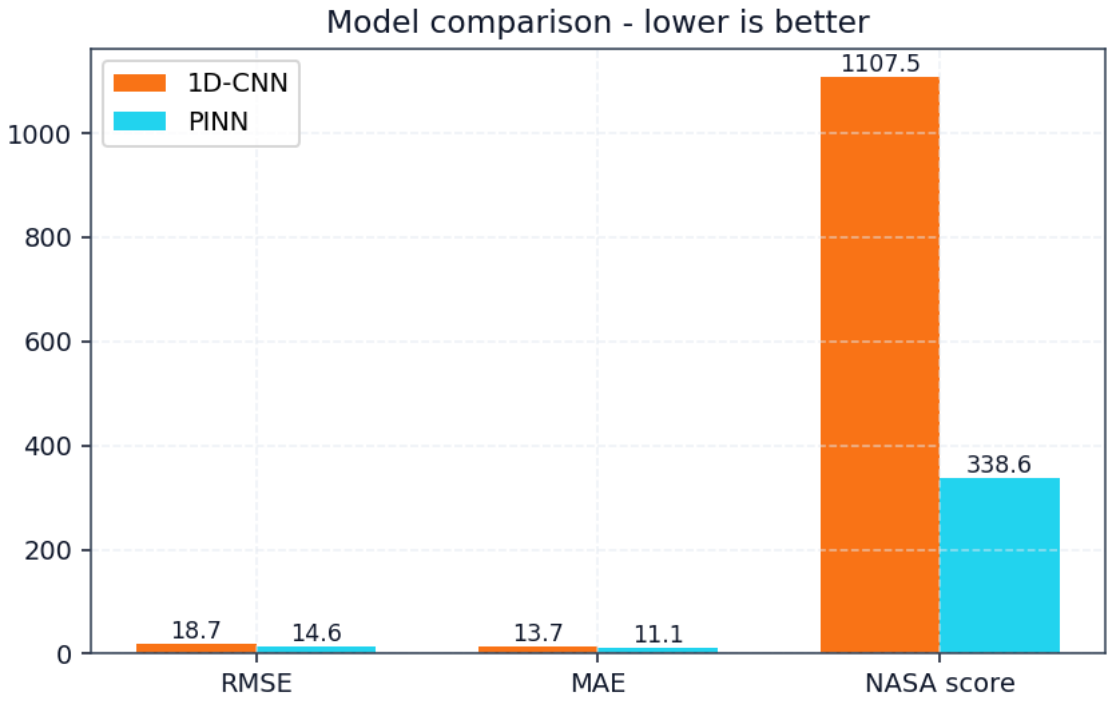
Why it matters for the research

The PINN scatter hugs the y=x diagonal much more tightly, and the end-of-life overshoot is largely gone. The monotonicity constraint in the loss has removed the optimistic bias that the CNN exhibits. This is the headline qualitative result of the project.

Talking point

'Same data, simpler network, right prior - and the pathological overshoot disappears.'

8. Quantitative Comparison - RMSE, MAE, NASA Score



What it shows

Bar chart comparing CNN and PINN across three metrics: Root Mean Square Error, Mean Absolute Error, and the asymmetric NASA scoring function.

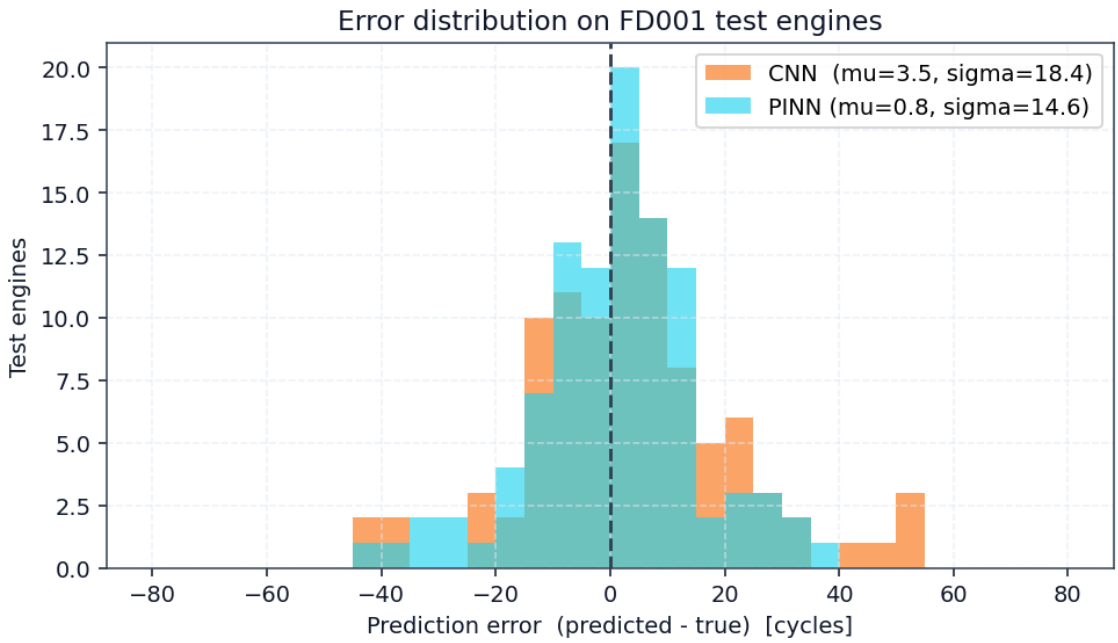
Why it matters for the research

RMSE drops from 18.68 to 14.64 (a 21.6 % improvement). MAE drops from 13.69 to 11.14 (18.6 %). But the NASA score - which penalizes late predictions roughly four times harder than early ones, matching the asymmetric real-world cost of missed warnings in aviation - collapses from 1107.5 to 338.6, a 69.4 % reduction. The physics prior does most of its work on exactly the errors that cost the most.

Talking point

'The 70 % reduction on the NASA score is the answer to why-should-anyone-care: this is the metric aviation maintenance actually uses.'

9. Error Distribution - Where the Mistakes Live



What it shows

Histogram of prediction errors (predicted RUL minus true RUL) on the test set, overlaid for CNN and PINN.

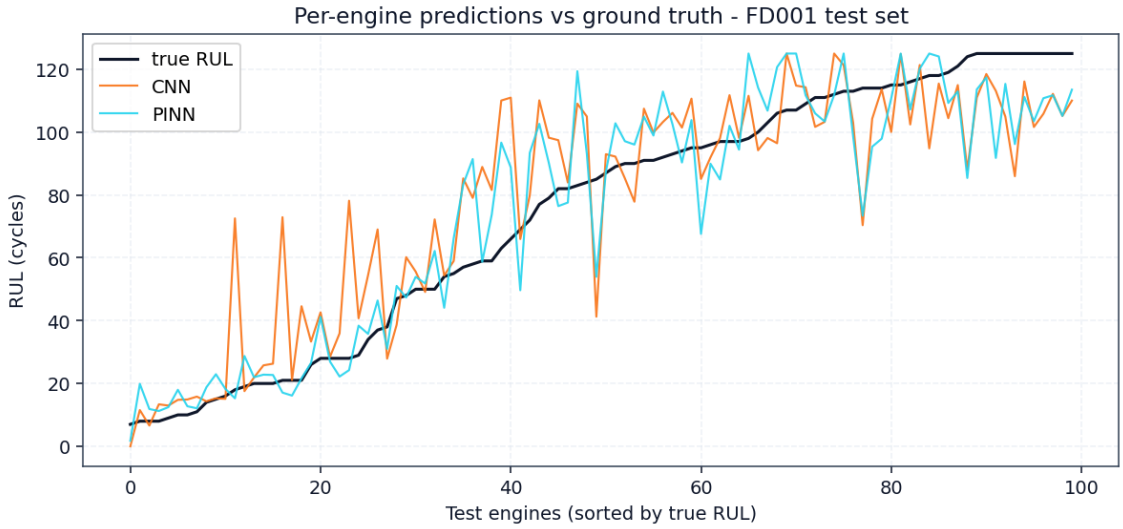
Why it matters for the research

The PINN error histogram is both narrower (lower variance) and more symmetric around zero. The right tail - positive errors, i.e. late predictions - is markedly thinner. This is the microscopic view of the NASA-score improvement from the previous plot: fewer large late errors means a smaller asymmetric penalty.

Talking point

'The PINN does not just reduce average error - it changes the shape of the error distribution. Fewer catastrophic late predictions.'

10. Per-Engine RUL Trajectories - Temporal Consistency



What it shows

For selected test engines, the predicted RUL trajectory from both models is plotted against the true linear decay, across the full sensor history of that engine.

Why it matters for the research

This plot validates that the physics constraint is doing what we designed it to do. CNN trajectories wander - they can predict rising RUL (physically impossible) and jitter between adjacent cycles. PINN trajectories are smooth and monotonically non-increasing, without any post-hoc smoothing or filtering. The monotonicity penalty in the loss is visibly shaping the output space.

Talking point

'No post-processing, no Kalman filter, no smoothing - the trajectories look physically sensible because the loss function made them that way.'

Key message for the defense

A smaller network with the correct physical prior outperforms a larger purely data-driven network. On the NASA C-MAPSS FD001 benchmark, embedding two simple physics constraints -- RUL must be monotonically non-increasing, and must decrease by exactly one per cycle -- cuts Root Mean Square Error by 21.6 % and the asymmetric NASA score by 69.4 %, without adding a single parameter.

The value of physics-informed learning is not in model capacity; it is in the shape of the solution. The CNN can, in principle, learn the same constraint from data, but at the cost of many samples and many parameters -- and with no guarantee. The PINN gets the constraint for free, from the problem specification.

Likely examiner questions

Q: Why not just use a larger CNN?
A: Size doesn't address the structural problem -- the CNN can still predict rising RUL. The physics loss eliminates that failure mode by design.

Q: How did you choose lambda values (0.15, 0.05)?
A: Held-out validation sweep. The drift penalty is weighted lower because the monotonicity penalty already enforces the sign; drift only fine-tunes the magnitude.

Q: Would this generalize to FD002-FD004?
A: The physics constraint is dataset-agnostic -- it only encodes the definition of RUL. We expect the absolute RMSE to rise with multi-mode faults, but the relative improvement of PINN over CNN should hold.

Q: Why pure NumPy and not PyTorch?
A: To emphasize that the method is implementation-light and reproducible without GPU access. 20.5 s of wall time on CPU makes the work transparent and verifiable.